09-Inversion of Control (IOC) pattern

- យើងចង់បានអី យើងអោយគេយកអោយយើង ដោយធ្វើការ Injection ហៅវាយកមកប្រើ
- IoC = the big idea (who controls): Control is taken away from your code and given to a framework/container យើងមិនចាំបាច់ធ្វើទេ Framework/ container វាជាអ្នកកំណត់អោយយើង។
- DI = the method (how it´s done): A way to achieve IoC by injecting dependencies from outside. ដើម្បីបានវាមក យើងត្រូវការអ្វីខ្លះ។

🍔 Easy Real-Life Example

Think about ordering a burger:

- **IoC** → You don't cook anymore, restaurant controls everything
- **DI** → The waiter brings the burger to your table

👉 Restaurant = IoC
👉 Waiter delivering food = DI

យើងគិត IoC ជាកន្លែងធំមួយ យើងចង់បានអី យកយើងយកវាមកប្រើ ដោយ Injection វា។

❌ Without IoC / DI

Java

```java
class Car {
    Engine engine = new Engine(); // you create it

    void drive() {
        engine.start();
    }
}
```

👉 You control everything
👉 No IoC, No DI

ឡានគឺជា IoC បើយើងត្រូវធ្វើការបើវា យើងត្រូវការបង្កើត Object Car នឹងមក ហើយហៅ engine.start()➔ មានន័យថា យើងជាអ្នកទៅបង្កើតវា មុនបានអាចយកវាមកប្រើប្រាស់បាន

✅ With IoC + DI

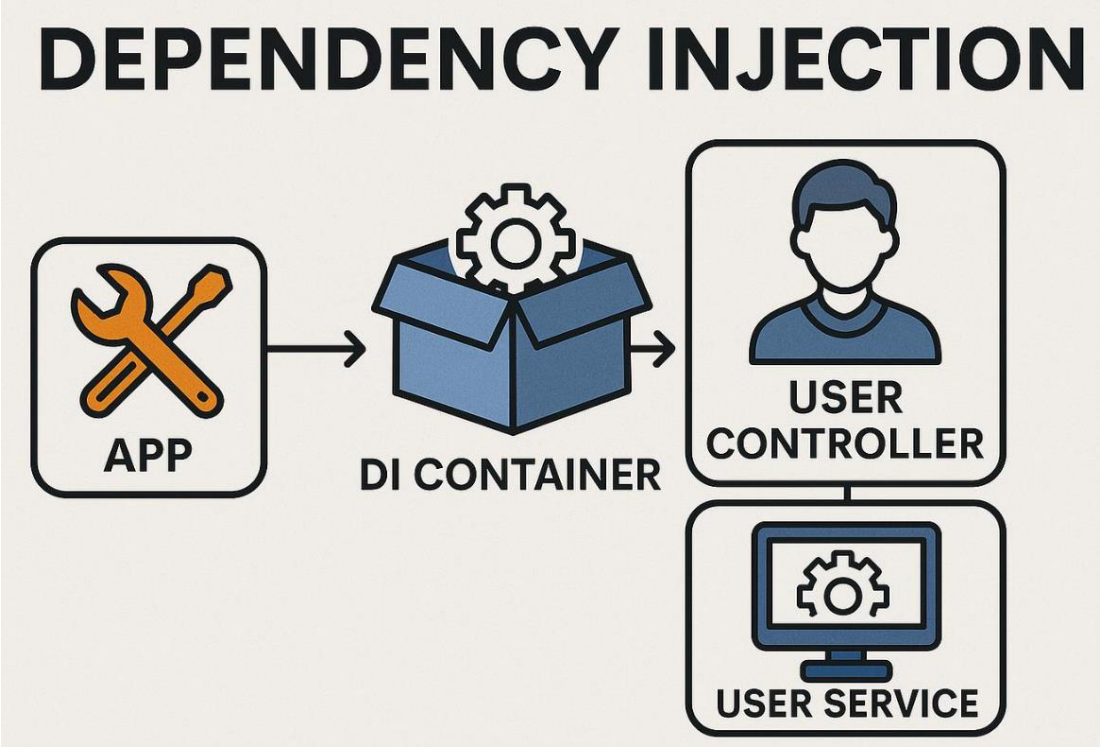
```java
class Car {
    private Engine engine;

    public Car(Engine engine) { // injected
        this.engine = engine;
    }

    void drive() {
        engine.start();
    }
}
```

👉 Someone else provides `Engine`

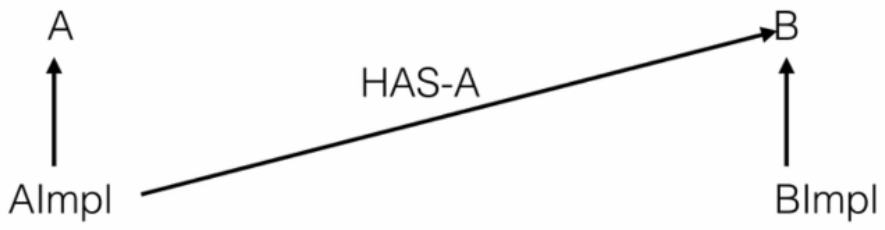
👉 That's DI

👉 And control is inverted → IoC

យើងចង់បើវា យើងគិតដល់ Container ដែលមានវា សិនថាមកពីណា ចឹងយើងអាចបង្កើត Constructor បានមុនអោយមិនចាំបាច់ បង្កើតអោយខ្លួនឯងទេ បន្ទាប់មកយើង អាចហៅវាមកប្រើ បាន ហើយ។



Dependency Injection Inversion Of Control



B b = new BImpl();

Spring Container

- Interface A has AImpl
- Interface B has Bimpl
- In the A need to create object from B to create Function
- DJ = will create the object for us no need we create by ourselves
- IC = Spring Container (where the Bimpl inside it)
- We can inject by money way : using a constructor, field level, setter methods.